



BEGINNER ONBOARDING GUIDE

How We Built the Review Intelligence Application

From a time-sensitive business question to a branded, public, interactive dashboard

WHAT THIS GUIDE DOES Explains the thinking, workflow, architecture, tools, and quality controls behind the application in plain language - so a beginner can reuse the approach for a different business problem.

CASE STUDY

Eight Kevin's Natural Foods beef HMR products | Public consumer reviews | GitHub Pages

Today, we're turning consumer feedback into focused action.
Because in the world we want tomorrow, better decisions begin with better evidence.

THE SHORT VERSION

The project in one page

The application began with a narrow business question: were consumer reviews validating a recent performance decline in eight beef home-meal-replacement products, and were texture complaints increasing? The work became valuable because the question stayed specific while the evidence and interface improved through iteration.

Phase	Objective	What happened in this project
1	Frame	Translate a business concern into explicit products, dates, questions, and decision needs.
2	Prototype	Use the available files to establish an initial signal and expose missing evidence.
3	Expand	Locate exact product pages, add retailers, and discover a complete first-party public review feed.
4	Prepare	Normalize fields, map exact products, remove duplicates, and separate strong evidence from context.
5	Analyze	Calculate trends, low-rating share, product performance, source mix, and transparent topic tags.
6	Design	Turn the analysis into a filterable application rather than a static report.
7	Publish	Host the static application on GitHub Pages with a stable organization-owned URL.
8	Validate	Run tests, inspect records, exercise filters, and verify the live public release.

What ultimately made it work

- A product registry fixed the scope before analysis began.
- Evidence layers were kept separate: dated full-text reviews, aggregate ratings, and assortment listings.
- The application was designed around questions users wanted to ask, not around a predetermined report layout.
- Every release was tested locally and again on the public URL.

BEGINNER PRINCIPLE Start with the smallest trustworthy version. Let gaps discovered in the first prototype tell you what to collect and build next.

PHASE 1 - FROM IDEA TO SCOPE

Turn the business question into a data contract

The original request was not 'build a dashboard.' It was a decision question: understand whether reviews had declined, whether the timing aligned with a recent performance concern, and whether texture complaints had increased.

The questions we wrote down first

- 1. Which products are actually in scope?
- 2. What date range defines the baseline and the recent period?
- 3. Which metrics would help a stakeholder decide where to investigate?
- 4. What evidence is strong enough to blend into a trend, and what should remain context only?
- 5. How will a non-analyst explore the underlying reviews?

The eight-product registry

A central registry became the source of truth for the exact products. It stored the canonical product name, aliases, public product handles, retailer URLs, UPCs where available, and pack-size distinctions. Every review had to map to one of these records before it could enter the analysis.

Registry field	Why it matters
Canonical product	The approved display name used throughout the dashboard.
Aliases	Alternative retailer or file names that resolve to the same product.
Exact identifiers	Product handles, UPCs, retailer page URLs, and provider IDs.
Pack-size rule	Exact 16 oz products could drive trends; Costco 32 oz club packs stayed contextual.
Exclusions	Chicken, Frozen Beef Bolognese, and other adjacent products were excluded.

WHY THIS MATTERS Most analysis failures begin before statistics - with ambiguous product matching. A small registry is one of the highest-value controls in a multi-source project.

PHASE 2 - MOVE INTO ACTION

Prototype first, then expand the evidence

The first version used the review files already available. That was enough to establish an initial directional signal and identify major gaps: limited retailer diversity, uneven product coverage, duplicate Amazon rows, and too little evidence for several products.

How the data sources evolved

Evidence wave	Contribution
Initial retailer files	Target and Amazon records provided the first dated review trend.
Retailer expansion	Kroger and Walmart pages added exact-product listings, review text where available, and aggregate rating snapshots.
Costco	Six same-flavor 32 oz club packs expanded assortment visibility but were not blended with exact 16 oz trends.
First-party discovery	The public storefront feed behind the eight Kevin's product pages exposed a complete, paginated review archive.
Final evidence base	593 retained records, including 588 metric-eligible full-text reviews and 422 first-party reviews.

The safe collection pattern

- Use public pages and public storefront feeds; do not bypass authentication or access controls.
- Capture the source URL, provider review ID, review date, rating, title, text, and available verification metadata.
- Paginate until the retrieved count reconciles with the page's displayed total.
- Filter only after capture so the original record can be audited.
- Respect platform terms, request rates, privacy, and the intended public use of the data.

IMPORTANT DISCOVERY The 2023 first-party volume spike reflected a different collection regime: many reviews carried store-invitation and future-purchase reward designations. That made collection method part of the analysis, not just metadata.

PHASE 3 - PREPARE TRUSTWORTHY DATA

Normalize, deduplicate, and label evidence quality

Each source represented reviews differently. The preparation layer converted them into one consistent schema and added explicit quality rules before any metric was calculated.

Normalized review schema

Field group	Examples
Identity	product_id, canonical product name, source, provider review ID
Observation	date, star rating, title, full review text
Provenance	capture type, source URL, verification status, transparency badges
Analysis control	metric_eligible flag and derived topic indicators

Three data-quality gates

1. Product gate: the record must map to one of the eight exact products.
2. Content gate: trend metrics require dated, full-text review evidence. Rating-only and analyst summaries remain context.
3. Duplicate gate: remove exact source-level duplicates and same-day cross-source duplicates using product, date, rating, title, and normalized text.

Why Amazon required extra caution

The Amazon export contained 600 raw rows and 60 unique rows associated with one variant-family listing. Some rows clearly discussed chicken despite appearing under the beef ASIN. Only 20 defensible Mongolian Style Beef records were retained; other variant-contaminated rows were excluded.

BEGINNER PRINCIPLE More rows are not automatically better data. Preserve provenance and make exclusions explainable; a smaller trustworthy dataset is more valuable than a larger ambiguous one.

PHASE 4 - ANALYSIS

Create simple metrics that answer the business question

The analysis layer stayed intentionally transparent. It used review counts, average rating, low-rating share, source mix, product comparisons, and keyword-coded topics that a stakeholder could understand and audit.

Analysis choice	Implementation
Default trend window	November 1, 2024 through July 15, 2026
Comparison point	Before versus on/after November 1, 2025
Core metrics	Review count, average rating, 1-2 star share, texture mentions, taste mentions
Portfolio result	Average rating moved from 3.09 to 2.29; low-rating share rose from 46.8% to 65.9%.
Priority products	Mongolian Style Beef and Sirloin Steak Tips with Gravy showed the strongest actionable risk.

Transparent topic coding

Regular-expression keyword rules tagged reviews for texture, taste/ flavor, sauce/ gravy, price/ portion, packaging, and product-changed language. Reviews could match more than one topic. The coding is directional and inspectable - not a claim that the model understands causality.

The collection-effect lesson

The 2023 first-party archive averaged 4.97 stars across 326 reviews, while the 2026 first-party records averaged 2.48 stars with 63% at 1-2 stars. Because the collection mechanism changed, that difference cannot be treated as a pure product effect. The dashboard therefore keeps 2023 searchable but defaults to a more recent trend window.

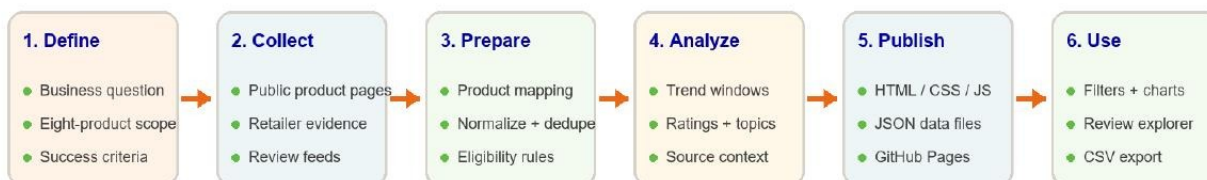
ANALYTICAL GUARDRAIL Consumer language indicates where to investigate. It does not prove a formulation, supplier, manufacturing, or lot-level cause without operational evidence.

PHASE 5 - APPLICATION ARCHITECTURE

Keep the system simple enough to understand

The dashboard is a static web application. It does not need a database server, application server, login service, or paid hosting layer. The browser downloads versioned JSON files and renders all filters, charts, tables, and review cards locally.

Static review-intelligence architecture



Quality loop

Validate records → run automated checks → inspect the live UI → refine scope, copy, and release

Repository structure

Component	Responsibility
<code>index.html</code>	Semantic page structure, filter controls, sections, and accessible labels.
<code>styles.css</code>	Mars colors, typography, responsive layout, cards, charts, and print behavior.
<code>app.js</code>	Loads JSON, manages filter state, computes views, renders charts/tables/reviews, and exports CSV.
<code>data/*.json</code>	Normalized reviews, analysis output, product registry, and retailer evidence.
<code>scripts/*.py</code>	Repeatable public-data collection and analysis preparation.
<code>tests.mjs</code>	Integrity assertions for counts, scope, sources, dates, snapshots, and prohibited language.

PHASE 6 - DESIGN AND ITERATION

Turn a report into an application

The first experience behaved like a report: it explained findings in a fixed order. Stakeholder feedback changed the design goal. The application needed to let users dissect the evidence themselves while still preserving recommended actions.

Key design iterations

Phase	Objective	What happened in this project
A	Scope clarity	Made the eight exact products visible and filterable everywhere.
B	Exploration	Added coordinated product, source, rating, topic, date, and text-search filters.
C	Evidence	Added a searchable review explorer and CSV export so every metric could be traced back to records.
D	Context	Separated retailer assortment, rating snapshots, and Costco club packs from dated trends.
E	Brand	Applied Mars colors, Arial/Mars Centra fallback typography, approved logo assets, and purpose-led copy.
F	Release	Added asset and data versioning after live QA exposed a mixed old/new GitHub Pages cache.

Beginner UI checklist

- Lead with the current filtered count so users always know what they are viewing.
- Make every chart respond to the same filters.
- Show sample size beside averages and percentages.
- Use plain-language labels and explain contextual evidence separately.
- Provide a path from summary metric to underlying records.
- Test the smallest screen size and keyboard-accessible controls.

PHASE 7 - QA AND PUBLISHING

Validate locally, publish, then validate again

Four layers of quality assurance

Layer	What to check
Record QA	Sample product assignments, dates, review content, source attribution, duplicates, and contamination terms.
Automated tests	Assert expected counts, eight-product scope, source set, date floor, snapshots, exclusions, and absence of internal/AI language.
Interaction QA	Exercise date modes, filters, sorting, review loading, source cards, product rows, CSV export, and responsive behavior.
Live QA	Open the public URL, verify release counts and controls, check console warnings, and confirm GitHub Pages serves coherent versioned assets.

How publishing worked

1. Create a clean Git repository containing only the public application and approved data.
2. Push the repository to the Mars-FN-Innovation-Prototypes GitHub organization.
3. Enable GitHub Pages from the main branch.
4. Use a stable organization-owned URL for stakeholder sharing.
5. Version JavaScript and data requests so cached files cannot create a mixed release.
6. Re-open the clean public URL and validate the production experience.

PUBLIC-RELEASE RULE Only publish information that is approved for public access. Keep credentials, private exports, personal data, internal notes, and raw restricted files out of the repository.

TOOLS USED

A practical, low-cost toolchain

Tool	Role in the workflow
AI-assisted workspace	Used to brainstorm the approach, inspect files/pages, draft and revise code, automate checks, and coordinate iteration. Human judgment defined scope, evidence rules, and release decisions.
Python	Collected public reviews, parsed storefront data, normalized text, deduplicated records, calculated metrics, and generated documents.
JSON	Portable data format connecting collection, analysis, testing, and the browser application.
HTML / CSS / JavaScript	Built the no-server interactive dashboard and branded stakeholder experience.
Git and GitHub	Version control, organization ownership, public repository hosting, and release history.
GitHub Pages	Free static hosting with a stable HTTPS public URL.
Browser automation	DOM-level interaction checks, console inspection, live release verification, and cache diagnosis.
Word / PDF tooling	Created branded executive summaries and onboarding artifacts with render-based visual QA.
Mars brand assets	Provided the approved palette, logos, typography direction, and purpose-led voice.

Why this stack is beginner-friendly

- Each layer can be opened and inspected with ordinary tools.
- The application has no runtime dependencies or server operating cost.
- JSON files make data changes visible and testable.
- Git preserves every change and enables safe collaboration.
- A static architecture is appropriate when the data updates in controlled batches rather than continuously.

REUSABLE PLAYBOOK

How to build a similar application next time

Before you build

- Write the decision question in one sentence.
- List the exact entities in scope and create a registry.
- Define the minimum trusted record and the contextual evidence layers.
- Choose the default time window and comparison point before looking at results.

During data work

- Capture provenance and provider identifiers with every record.
- Keep raw capture separate from normalized analysis data.
- Reconcile pagination totals and document exclusions.
- Test for duplicates, variant contamination, source bias, and collection-method changes.

During application work

- Design filters around stakeholder questions, not available columns.
- Show sample sizes and make records inspectable.
- Keep contextual evidence visually separate from trend-driving evidence.
- Version data and assets together as one release.

Before sharing

- Run automated checks and a manual record sample.
- Test the public URL in a clean browser session.
- Confirm there are no credentials, personal data, internal notes, or unapproved claims.
- Prepare a short executive summary that states findings, limitations, and recommended actions.

BEST ADVICE Do not wait for perfect data before building a useful prototype. Do insist on explicit scope, traceable records, and honest limitations before making the prototype public.

GLOSSARY AND HANDOFF

Key terms for a beginner

Term	Plain-language definition
API / feed	A structured way for one system or webpage to expose data to another program.
Backfill	Loading historical records so the application can analyze earlier periods.
Canonical product	The single approved identity used to combine aliases from multiple sources.
Client-side	Work performed in the user's browser rather than on a web server.
Deduplication	Identifying and removing repeated copies of the same record.
First-party review	A consumer review published on Kevin's own product page rather than a retailer page.
Git	A system that records changes to files and code over time.
GitHub Pages	A GitHub service that publishes static repository files as a website.
HMR	Home meal replacement - a prepared or nearly prepared meal intended for convenient at-home eating.
JSON	A readable text format for structured data used by both Python and JavaScript.
Provenance	Information showing where a record came from and how it was captured.
Static application	A website made of browser files that does not require a continuously running backend server.

Where to look in this project

Public application: <https://mars-fn-innovation-prototypes.github.io/knf-beef-review-dashboard/>

Repository: <https://github.com/Mars-FN-Innovation-Prototypes/knf-beef-review-dashboard>

Start with README.md, then inspect index.html, app.js, tests.mjs, data/, and scripts/ in that order.

HANDOFF OUTCOME A new contributor should be able to explain the evidence model, rerun the collector and analysis scripts, make a controlled UI change, run tests, and publish a coherent release without changing the underlying business scope.